# Lab 4 - Candidate moves

Author: Michał Żurawski 160252

## Description of the problem

Each instance contains nodes with coordinates (x, y) and a cost. Select exactly half of the nodes (rounded up) and connect them into a closed cycle. Minimise **cycle length + sum of the costs of the selected nodes**. Distances are Euclidean, rounded to the nearest integer.

## Algorithms

One method, 200 runs: steepest local search with lab 3's best neighborhood (inter-route node exchange + edges exchange) from random starts (run i uses seed 42 + i, so the same starting tours as lab 3), but each step evaluates only **candidate moves**, the moves that add an edge to one of the 10 nearest nodes. The baseline is lab 3's steepest LS, edges exchange, random start, which evaluates the whole neighborhood.

### Intuition

Steepest local search spends almost all its time on hopeless moves: in a good tour nearly every edge joins a node to one of its few nearest nodes, so a move that adds an edge to a far node must remove even longer edges just to break even, and that almost never happens. Candidate moves skip them: for every node we keep a short list of its nearest nodes, and evaluate only moves that bring one of these short edges into the tour. Example: one step of the full search evaluates $100 \cdot 100 + 100 \cdot 99 / 2 = 14\,950$ moves; the candidate search evaluates at most $100 \cdot 10 \cdot 2 = 2\,000$, 7.5× fewer. The improving moves it skips are the price paid in quality: small on TSPB, 5 % on TSPA.

### Core calculations

Candidate edges: N(a) = the 10 nodes $b \neq a$ with the smallest

`d(a,b) + c(b)`

and (a, b) is a candidate edge if $b \in N(a)$. p(x), s(x) = predecessor, successor of x in the tour.

For a in the tour and $b \in N(a)$, the two moves that add (a, b):

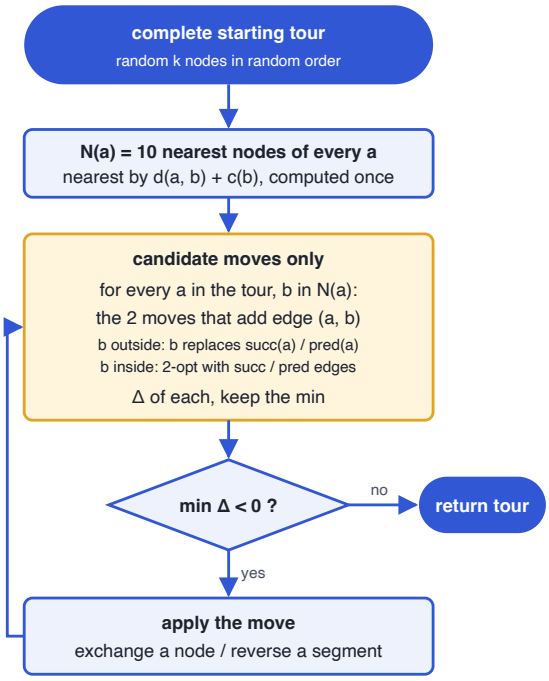
b not in the tour, b replaces s(a) (or symmetrically p(a)):

`Δ = c(b) − c(s) + d(a,b) + d(b,s(s)) − d(a,s) − d(s,s(s))`

b in the tour, edges exchange with the successors (or symmetrically the predecessors):

`Δ = d(a,b) + d(s(a),s(b)) − d(a,s(a)) − d(b,s(b))`

The move with the lowest Δ is applied if $\Delta < 0$; otherwise the tour is a local optimum.




| Method | moves per step |
|---|---|
| Baseline (lab 3) | 14 950 |
| Candidate moves | ≤ 2 000 |

### Pseudocode

Simplified from `python/solver.py`. `local_search` is lab 3's steepest loop; only the move search changed: `find_best_candidate_move` generates the moves from the candidate lists instead of scanning a fixed list of all moves, so `position[v]` (index of v in the tour, or `NOT_SELECTED`) is kept up to date by `apply_move`.

```
def solve(seed):
    candidates = nearest_candidates()           # N(a) for every node a
    tour = random_start(rng(seed))              # same start as lab 3
    return local_search(tour, candidates)

def local_search(tour, candidates):
    position = tour_positions(tour)
    while True:
        move = find_best_candidate_move(tour, position, candidates)
        if move is None:
            return tour                         # local optimum
        apply_move(tour, position, move)

def find_best_candidate_move(tour, position, candidates):
    best_move, best_delta = None, 0
    for i, a in enumerate(tour):
        for b in candidates[a]:
            for move in moves_adding_edge(tour, position, i, b):
                delta = move_delta(tour, move)
                if delta < best_delta:
                    best_move, best_delta = move, delta
    return best_move

def moves_adding_edge(tour, position, i, b):
    successor, predecessor = i + 1, i - 1         # modulo k
    j = position[b]
    if j == NOT_SELECTED:                         # b enters next to a
        return [(EXCHANGE_SELECTED, successor, b), (EXCHANGE_SELECTED, predecessor, b)]
    if j == successor or j == predecessor:
        return []                                 # edge (a, b) already in the tour
    return [edges_exchange(i, j),                 # adds (a, b) and (succ a, succ b)
            edges_exchange(predecessor, j - 1)]  # adds (pred a, pred b) and (a, b)

def apply_move(tour, position, move):
    kind, i, j = move
    if kind == EXCHANGE_SELECTED: tour[i] = j, update position of the old and new node
    if kind == SWAP_EDGES:        reverse tour[i+1 .. j], update positions in it
```

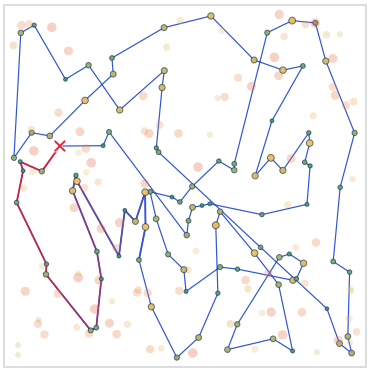
**Why only these moves.** An exchange that replaces a itself with b would not create edge (a, b), so b replaces a neighbour of a instead. Each candidate edge can be added by exactly two moves, both of one kind: two node exchanges if b is not in the tour, two edges exchanges if it is. They are generated directly, so no move outside the candidate set is ever looked at.

## Results

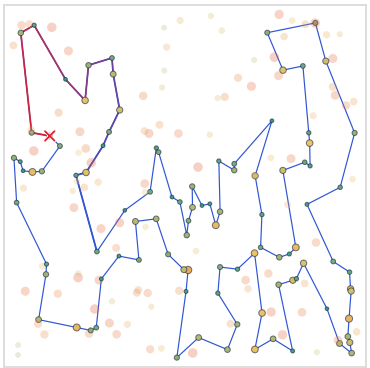
Objective = cycle length + sum of node costs, over 200 runs per method. Each cell is av (min – max); the best value of each kind per instance is in bold.
All best solutions were checked with the solution checker (the same computation as the lecturer's Solution checker.xlsx): **yes**.
Results are identical in all implemented languages: **yes**.

| Method | TSPA | TSPB |
|---|---|---|
| Steepest LS, nodes exchange, random start | 89742 (81937 – 96386) | 66055 (59059 – 74933) |
| Steepest LS, edges exchange, random start | **76225** (**74248** – **79388**) | **52461** (50265 – **55173**) |
| Steepest LS, edges exchange, candidate moves, random start | 80062 (76007 – 85627) | 52560 (**49902** – 55299) |

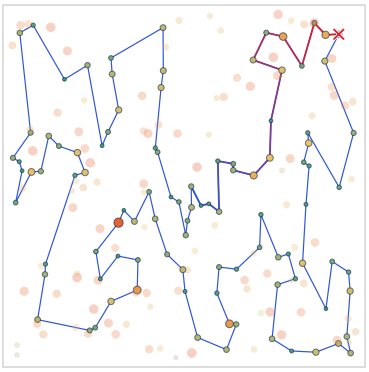
### Visualisation (Instance TSPA)



Steepest LS, nodes exchange, random start
**81937** (length 30984 + cost 50953)

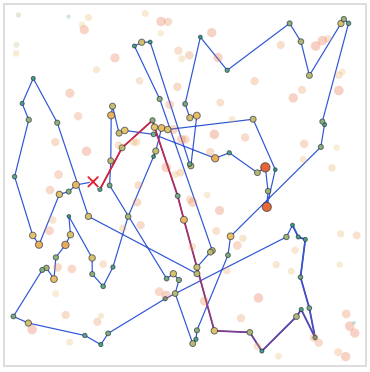


Steepest LS, edges exchange, random start
**74248** (length 22259 + cost 51989)

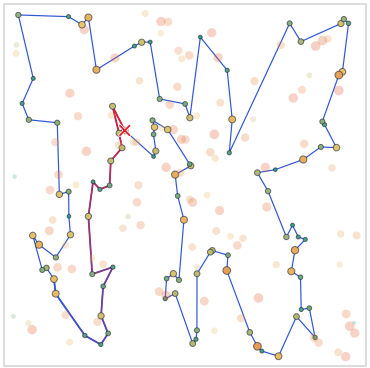


Steepest LS, edges exchange, candidate moves, random start
**76007** (length 22323 + cost 53684)

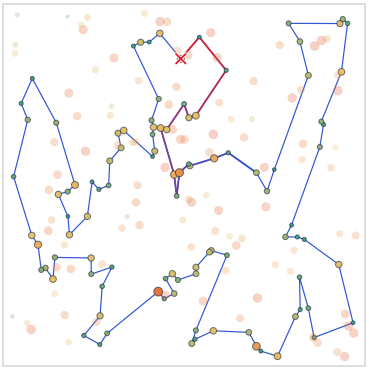
### Visualisation (Instance TSPB)



Steepest LS, nodes exchange, random start
**59059** (length 28949 + cost 30110)



Steepest LS, edges exchange, random start
**50265** (length 19741 + cost 30524)



Steepest LS, edges exchange, candidate moves, random start
**49902** (length 19786 + cost 30116)

## Running time

Average time of one run of the algorithm only, over the same runs as above. The fastest language in each row is in bold.

### Instance TSPA

| Method | Python | C | C++ | Rust |
|---|---|---|---|---|
| Steepest LS, nodes exchange, random start | 336.9 ms | 4.69 ms | **4.31 ms** | 6.62 ms |
| Steepest LS, edges exchange, random start | 255.2 ms | 3.48 ms | **3.38 ms** | 4.13 ms |
| Steepest LS, edges exchange, candidate moves, random start | 54.18 ms | **0.6827 ms** | 1.24 ms | 1.03 ms |

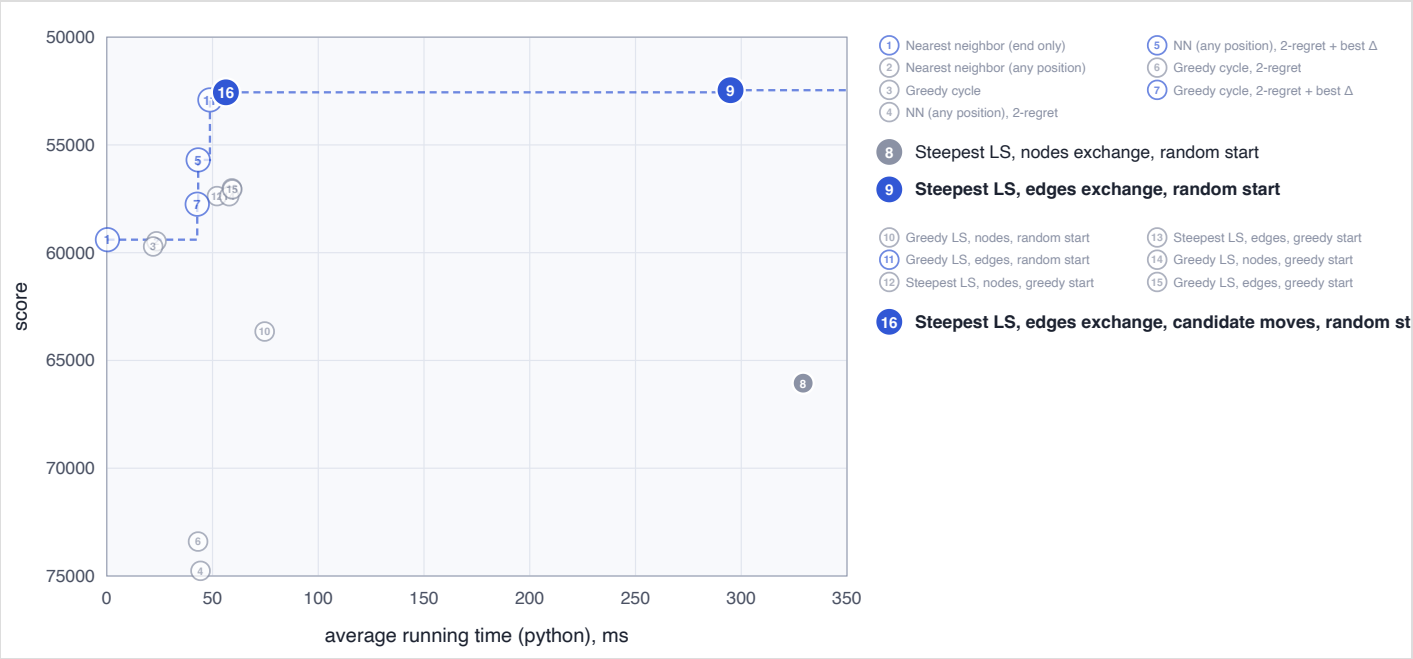
### Pareto plot (Instance TSPA)



## Instance TSPB

| Method | Python | C | C++ | Rust |
|---|---|---|---|---|
| Steepest LS, nodes exchange, random start | 329.3 ms | 4.57 ms | **4.37 ms** | 5.46 ms |
| Steepest LS, edges exchange, random start | 294.9 ms | 3.54 ms | **3.44 ms** | 4.28 ms |
| Steepest LS, edges exchange, candidate moves, random start | 56.34 ms | **0.6627 ms** | 1.17 ms | 0.9814 ms |

## Pareto plot (Instance TSPB)



## Best solutions (0-based node indices)

### Instance TSPA

**Steepest LS, edges exchange, candidate moves, random start**
55, 17, 29, 178, 24, 136, 186, 184, 164, 142, 194, 54, 9, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 139, 112, 115, 157, 71, 4, 76, 132, 91, 80, 48, 31, 60, 116, 149, 63, 168, 11, 49, 108, 152, 100, 90, 123, 97, 127, 69, 171, 2, 189, 27, 87, 141, 187, 30, 191, 180, 28, 35, 185, 45, 73, 135, 181, 192, 158, 20, 58, 84, 134, 176, 174, 150, 106, 89, 151, 182, 57, 64, 88, 78, 153, 170, 159, 130, 107, 113, 93, 50, 3, 169, 1, 47, 147, 101

### Instance TSPB

**Steepest LS, edges exchange, candidate moves, random start**
156, 89, 57, 86, 46, 72, 117, 8, 122, 151, 173, 184, 3, 135, 76, 162, 192, 54, 187, 70, 190, 81, 94, 105, 61, 196, 114, 10, 127, 80, 129, 90, 118, 110, 2, 14, 153, 20, 74, 17, 171, 39, 99, 88, 87, 82, 179, 67, 24, 34, 152, 181, 167, 73, 59, 51, 85, 107, 45, 134, 175, 157, 60, 40, 71, 155, 6, 197, 29, 132, 103, 178, 65, 145, 28, 149, 158, 26, 125, 177, 170, 115, 97, 154, 140, 47, 38, 21, 30, 183, 69, 160, 172, 136, 133, 120, 121, 198, 96, 48

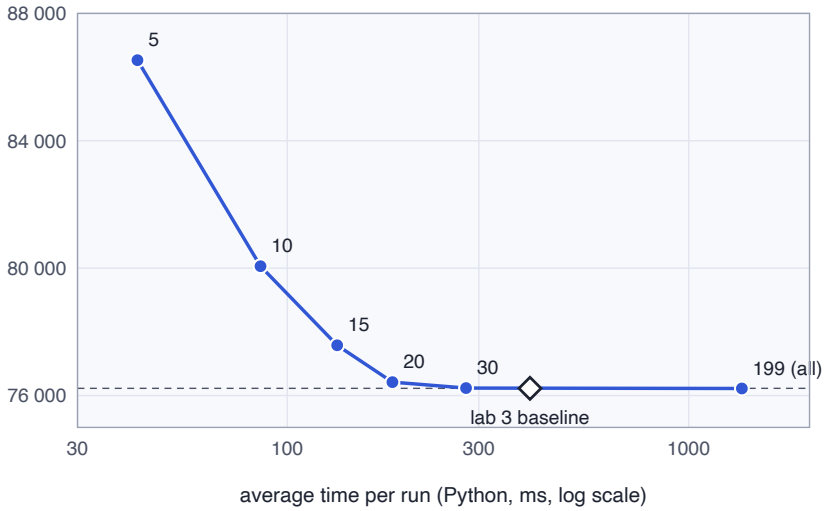
## Source code

labs/lab4 (Python, C, C++, Rust)

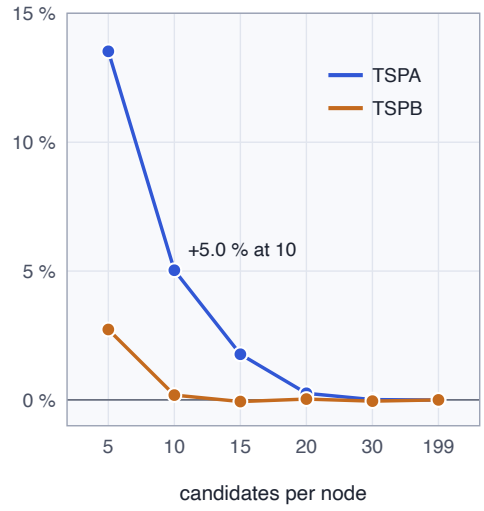
## Number of candidates (extra experiment)

The assignment fixes 10 candidates but allows choosing it experimentally. Same 200 random starts, Python implementation, objective av (min – max) and average time per run (timings come from a separate run, so the 10-candidate time differs slightly from the table above):

| Candidates | TSPA | TSPB | time TSPA | time TSPB |
|---|---|---|---|---|
| 5 | 86531 (80176 – 91338) | 53894 (51266 – 56792) | 42 ms | 46 ms |
| **10** (reported above) | 80062 (76007 – 85627) | 52560 (49902 – 55299) | 86 ms | 94 ms |
| 15 | 77578 (74096 – 81478) | 52429 (50177 – 55464) | 133 ms | 143 ms |
| 20 | 76423 (74099 – 80271) | 52480 (50265 – 54993) | 183 ms | 189 ms |
| 30 | 76235 (74248 – 79270) | 52437 (50265 – 55410) | 279 ms | 265 ms |
| 199 (all nodes) | 76224 (74248 – 79388) | 52461 (50265 – 55410) | 1357 ms | 1294 ms |
| baseline (lab 3) | 76225 (74248 – 79388) | 52461 (50265 – 55173) | 402 ms | 356 ms |




average time per run (Python, ms, log scale)




candidates per node

## Conclusions

- Candidate moves cost little quality on TSPB (52 560 vs 52 461 on average) but 5 % on TSPA (80 062 vs 76 225), where the improving moves more often need an edge outside the 10 nearest nodes; 20 candidates close that gap.
- Evaluating at most 2 000 instead of 14 950 moves per step makes steepest LS 2.7–5.3× faster depending on the language (C: 0.68 ms vs 3.48 ms on TSPA, Python: 54 ms vs 255 ms).
- The best single solution of TSPB so far comes from candidate moves (49 902, below 49 955 of greedy LS in lab 3).